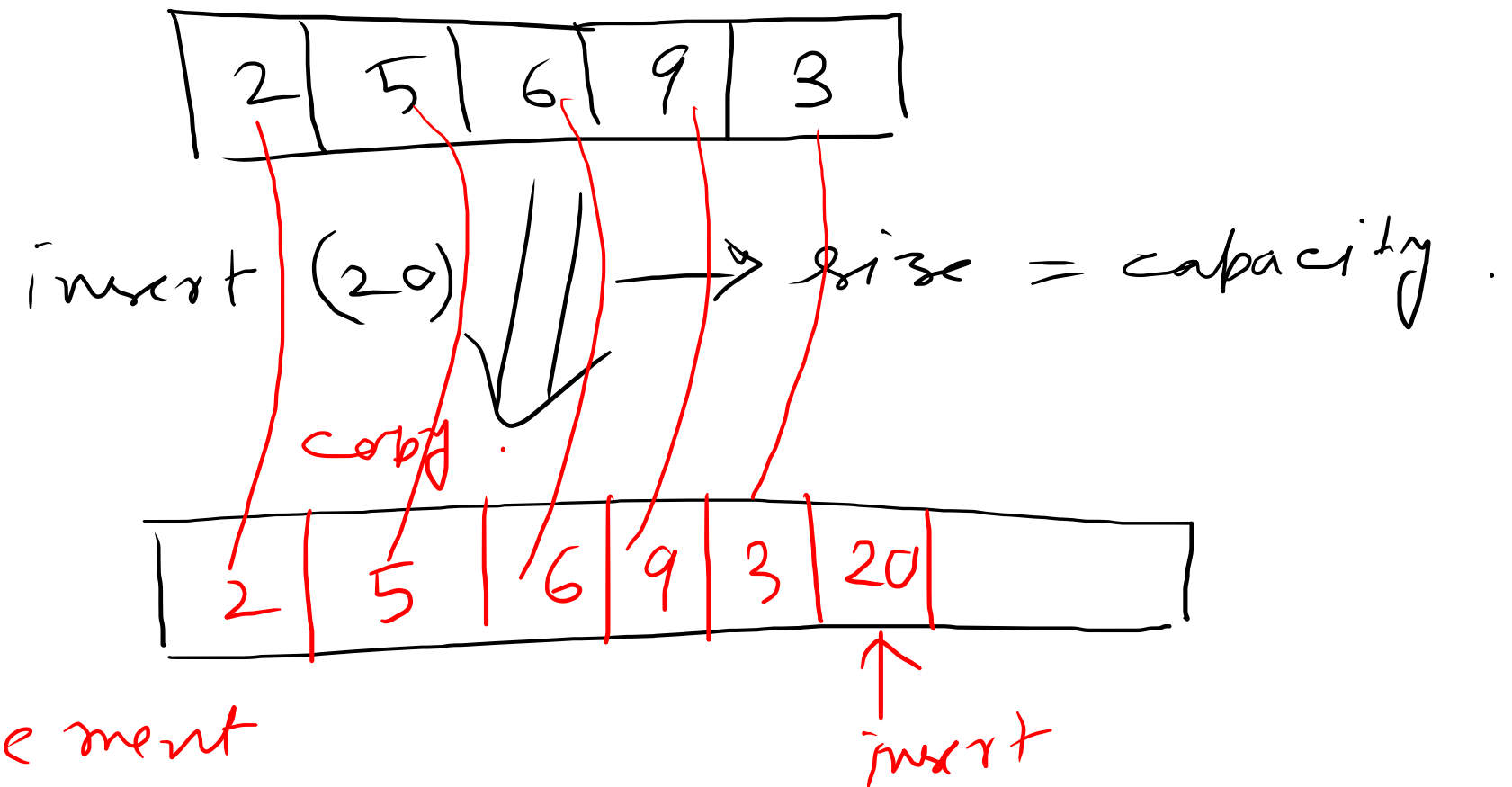


Dynamic array / Table doubling.

Insert



Q: How much increment we make?

i) new capacity = old-capacity + 1 $\rightarrow$ amortized cost per operation is $O(1)$

ii) new capacity = old capacity + $\underset{\substack{\downarrow \\ \text{Constant}}}{K}$ $\rightarrow O(1)$

iii) new capacity = old-capacity + old-capacity.
= $2 \times$ old-capacity.

	capacity	size	cost
	0	0	0
Insert.	1	1	c
insert.	2	2	→ c + c
insert.	4	3	→ 2c + c
"	4	4	c
	8	5	→ 4c + c
	8	6	c
	8	7	c
	8	8	c
	16	9	→ 8c + c

Total cost

$$\leq \sum_{i=1}^n c + \sum_{i=0}^K 2^i c$$

$i = 1, 2, 3, \dots$

$K = \log_2 n$

$$\rightarrow cn + c(2^{K+1} - 1)$$

$$= cn + 2c2^K - c$$

$$= cn + 2cn - c$$

$$= 3cn$$

Amortized cost
per operation: $\frac{3cn}{n}$
 $= O(1)$

$$n = 16$$

n

4

$$K = 4$$

Flow

look for some fractional value.

$$\text{new-capacity} = \alpha \cdot \text{old capacity}$$

$$\alpha > 1$$

Deletion

A:

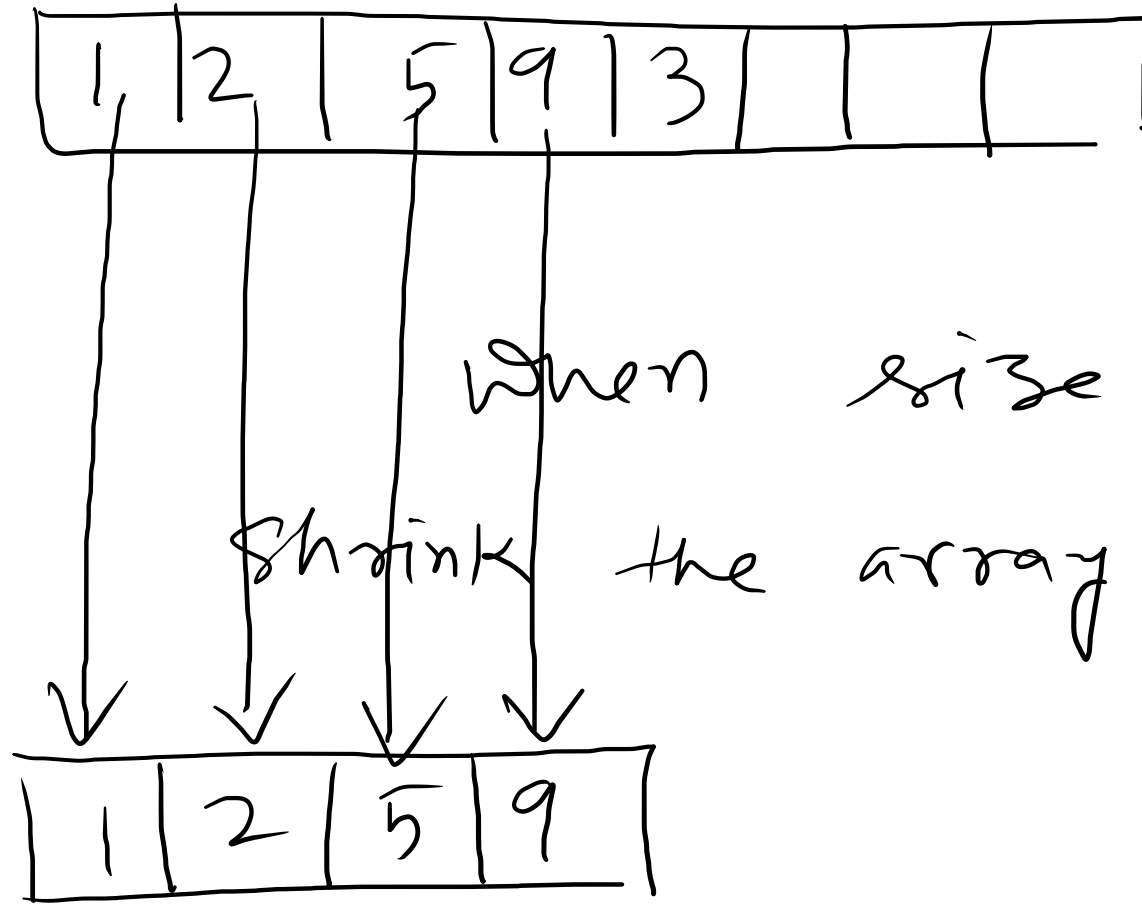
2	5	3	1	6	9	4	5
---	---	---	---	---	---	---	---

Delete in a dynamic array
has no problem.

2	5	3	1				
---	---	---	---	--	--	--	--


space

Delete (3)



- i) $\text{new} = \text{old} - 1$
- ii) $\text{new} = \text{old} - 1$
- iii) $\text{new} = \text{old} - \frac{1}{2} \text{old}.$

i)

	capacity	size	cost
	64	64	0
delete	63	63	$c + 63c$
delete	62	62	$c + 62c$

total cost: $O(n^2)$

ii)

Delete

capacity

64

64

⋮

64-K

64-2K

,

⋮

⋮

⋮

⋮

size

64

63

64-K

⋮

64-2K

cost

0

c

⋮

(64-K)c + c

c

⋮

(64-2K)c + c

Total cost: $O(n^2)$

iii) Halving

	capacity	size	cost
	64	64	0
delete	64	63	c
.	.	.	.
.	64	33	c
.	32	32	$32c + c$
.	.	.	c
.	.	.	c
.	.	.	.
.	.	.	.
.	16	16	$16c + 1$
.	.	.	.
.	.	.	.
.	.	.	.
			total cost! $O(n)$

mix of n delete and insert operation.

2	5	9	6	3	1	2	
---	---	---	---	---	---	---	--

capacity 8
size 7.

Del

2	5	9	6	3	1		
---	---	---	---	---	---	--	--

— c

Del

2	5	9	6	3			
---	---	---	---	---	--	--	--

— c

Del

2	5	9	6				
---	---	---	---	--	--	--	--

=>

2	5	9	6
---	---	---	---

— 4c + c

insert (8)

2	5	9	6	8			
---	---	---	---	---	--	--	--

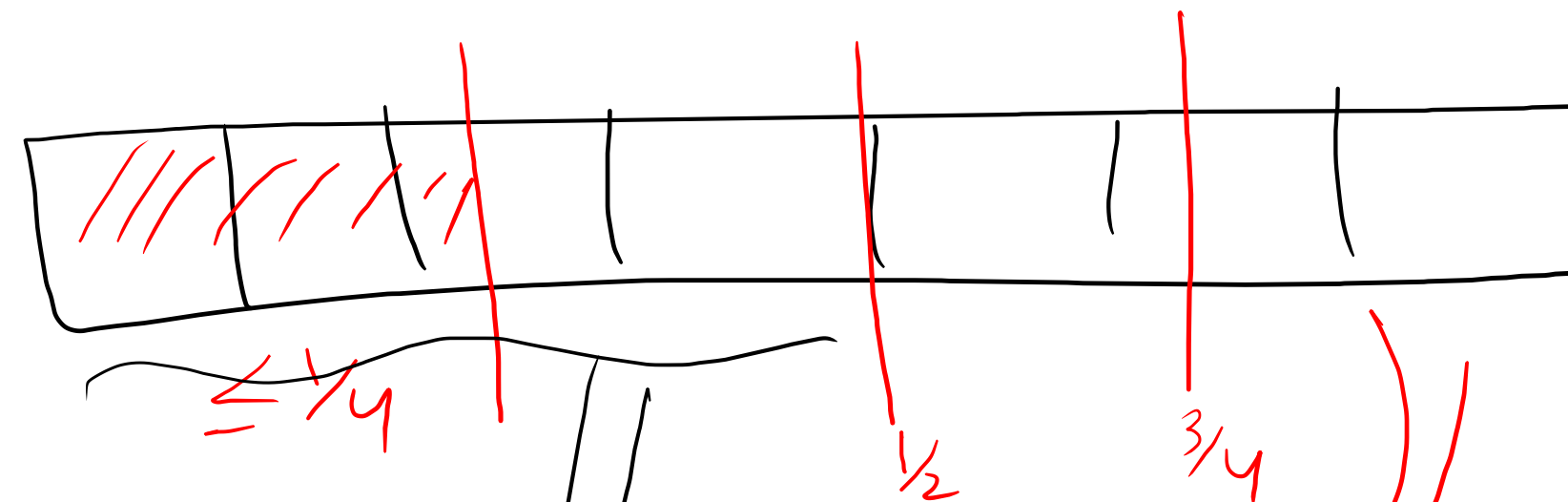
—> 4c + c

Del

insert

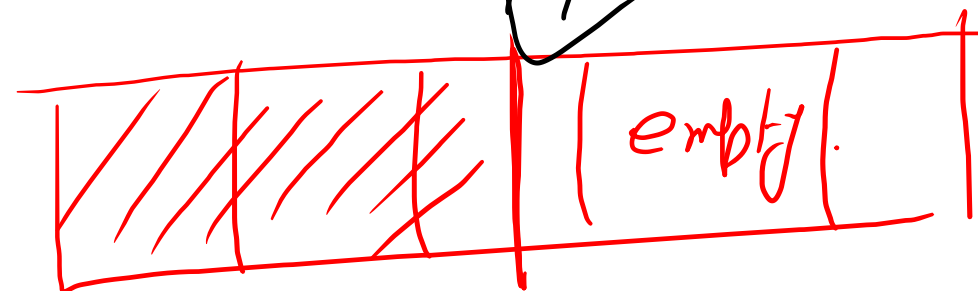
Del

insert



When size $\leq 1/4$ of capacity -

Shrink



min of insert and delete.

for insertion: $\text{size} = \text{capacity}$ then double

for deletion: $\text{size} << \text{capacity}$ then halving.
 $\text{size} \leq \frac{1}{4} \text{ capacity}$

	capacity	size	cost
	4	20	0
Delete	6	19	C
.	6	18	.
.	6	17	C
Delete	32	16	$16C + C$
insert	32	17	C
.	.	18	.
.	32	19	.
.	.	18	C
Delete	32	18	C
.	.	.	.
.	.	.	.
.	.	.	.

8

Total cost:
 $O(n)$

Python : - List $\Leftarrow \approx 1.2$

Java : ArrayList $\Leftarrow \approx 1.5$

C++ : Vector $\Leftarrow \approx 1.5 - 2$

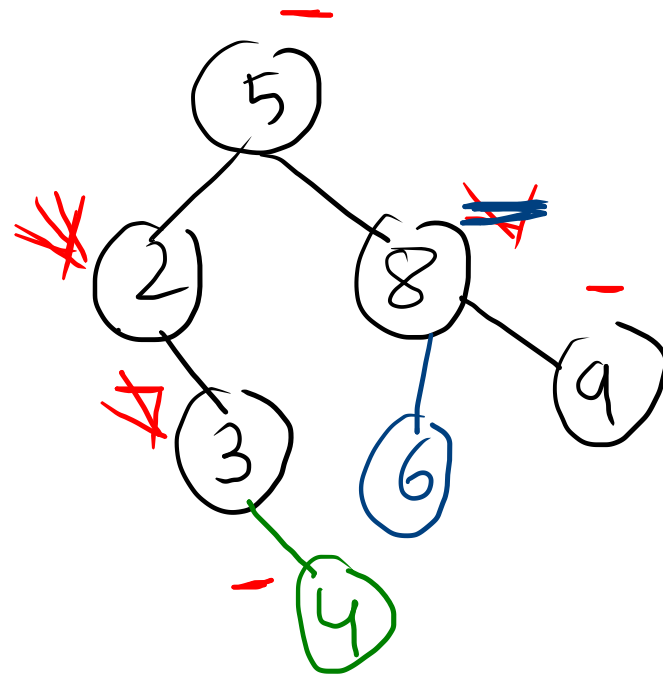
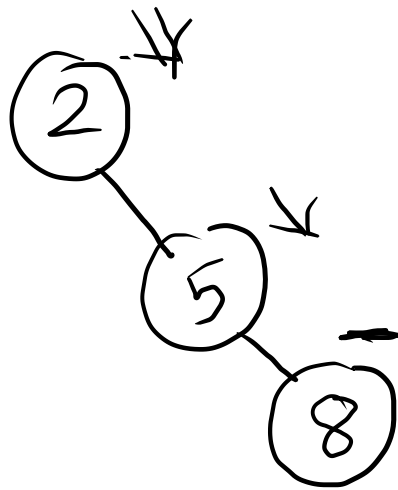
Amortized analysis

- └ Aggregate analysis. (we have done)
- └ Accounting method
- └ Potential method.

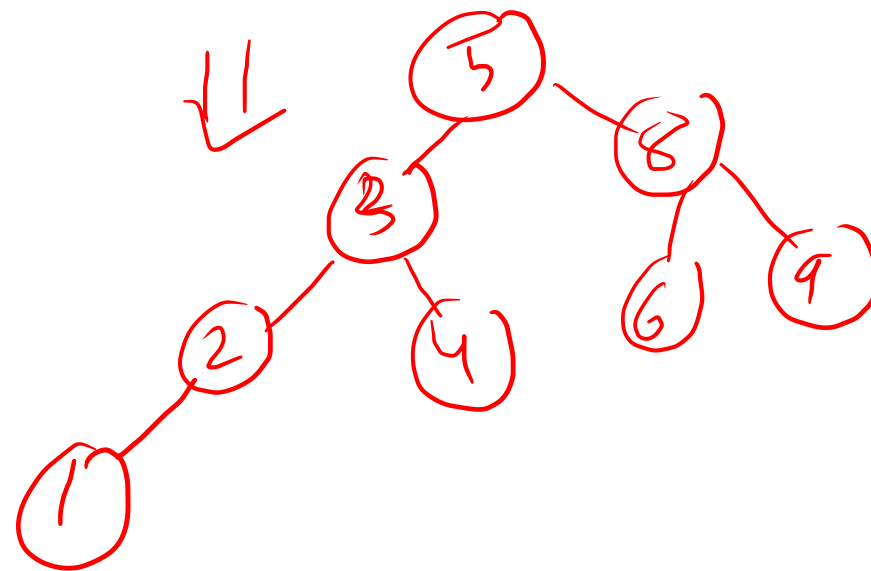
Red black tree

Insert

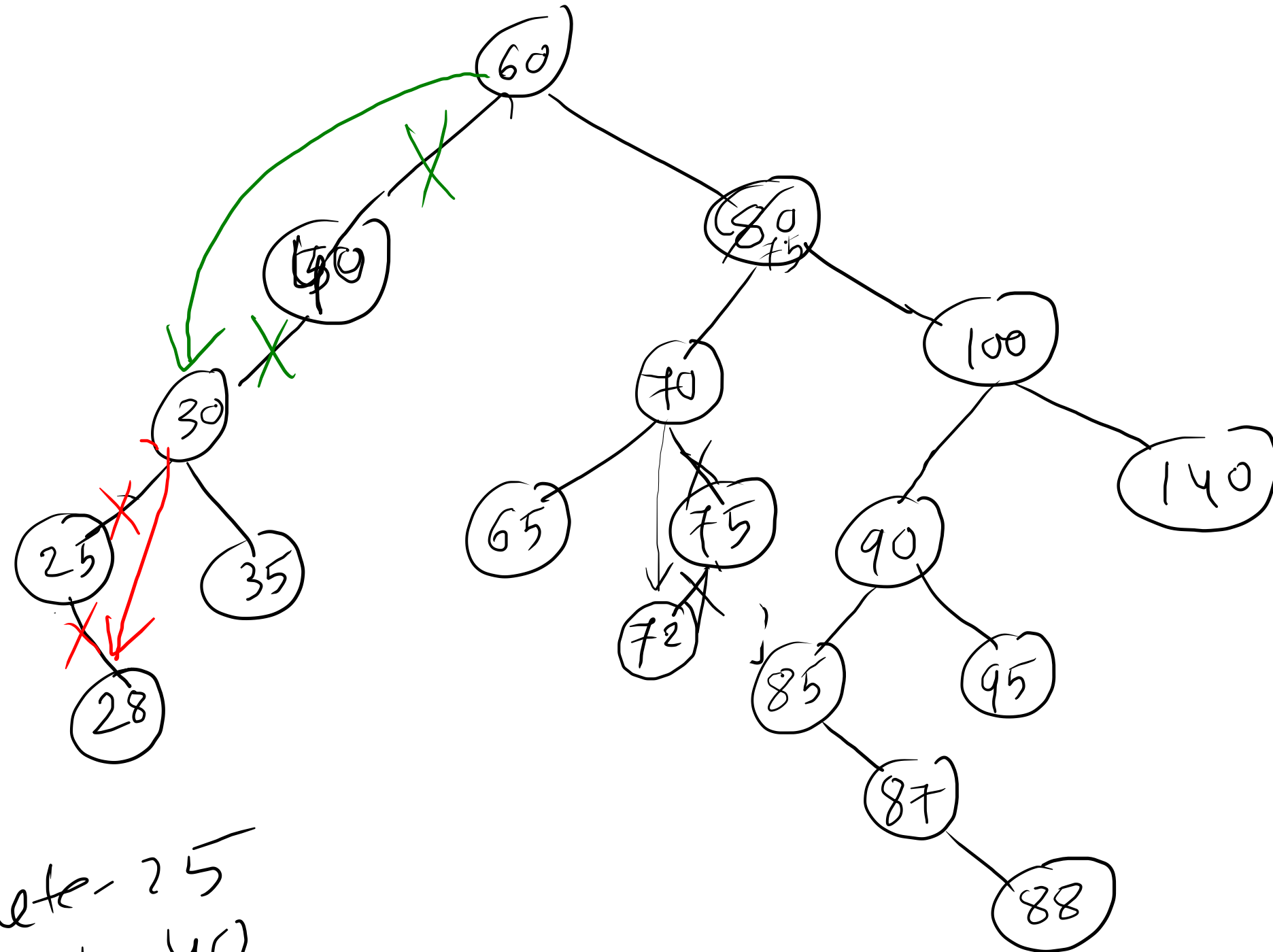
2, 5, 8, 3, 9, 6, 4, 1



left
right



Binary Search Tree



✓ Delete - 25
Delete 40
Delete - 80